

Full-Stack Project Documentation

A complete guide to the Frontend & Backend architecture,
API design, database schema, and component structure.

React 18 | Vite | TailwindCSS | Node.js | Express 5
MySQL | JWT | Razorpay | bcryptjs

May 2026

Table of Contents

1. Project Overview

- Purpose & Goals
- Technology Stack
- Feature Summary

2. System Architecture

- High-Level Architecture
- Request Flow
- Authentication Flow

3. Backend

- Project Structure
- Server & Configuration
- Database Models & Schema
- Middleware
- API Endpoints — Auth
- API Endpoints — Jobs
- API Endpoints — Community
- API Endpoints — Resume
- API Endpoints — Payments

4. Frontend

- Project Structure
- Routing & Page Map
- Auth Context & State Management
- API Service Layer
- Pages — Detail
- Components

5. Subscription & Payments

- Free vs Premium
- Razorpay Integration Flow

6. Environment Variables

7. Setup & Running Locally

1. Project Overview

Jobify is a full-stack web application that helps job seekers track their applications, manage resumes, and share interview experiences with a community. It features a subscription model powered by Razorpay, allowing free users to upgrade to a premium tier that unlocks unlimited community access and advanced features.

Purpose & Goals

- Allow users to log and manage job applications end-to-end (saved → applied → interviewing → offer/rejected).
- Provide a dashboard with real-time stats across all application statuses.
- Let users build and export a formatted resume directly from the platform.
- Enable community knowledge-sharing: users can publish their interview questions and experiences.
- Gate premium community features behind a one-time ₹500 Razorpay payment.

Technology Stack

Layer	Technology	Notes
Frontend	React 18 + Vite	SPA with client-side routing
Styling	TailwindCSS	Utility-first CSS
HTTP Client	Axios	Interceptors for auth token injection
Auth State	React Context API + jwtDecode	Token stored in localStorage
Backend	Node.js + Express 5	ES Modules (import/export)
Database	MySQL via mysql2/promise	Connection pool
Auth	JWT + bcryptjs	7-day token, hashed passwords
Payments	Razorpay SDK	Order create + HMAC verification
Resume Export	html2pdf.js	Client-side DOM-to-PDF

Feature Summary

Job Tracker	Add, edit, delete jobs. Track status: saved, applied, interviewing, offer, rejected.
Dashboard	Metric cards showing counts per status. Recent jobs list.
Community	Public feed of interview Q&A. Premium users see all posts; free users see 5.
Resume Builder	Fill in personal info, skills, education, projects, experience. Download as PDF.
Payments	Razorpay checkout for ₹500 one-time upgrade. Signature HMAC verified server-side.
Auth	Register / Login with JWT. Protected routes in frontend and backend.
Profile	View account info and subscription tier.

2. System Architecture

High-Level Architecture

Jobify follows a classic **client-server architecture**. The React SPA runs in the browser and communicates with an Express REST API over HTTP. The API connects to a MySQL database using a connection pool for efficiency.



Request Flow

Every authenticated API request follows this lifecycle:

1	User action	User interacts with a page (e.g., clicks 'Add Job').
2	Axios call	The frontend api.js service sends an HTTP request. The request interceptor automatically attaches the JWT from localStorage as a Bearer token.
3	Auth middleware	authMiddleware.js extracts and verifies the JWT. If invalid or missing, returns 401.
4	Controller	The relevant controller function runs the business logic and calls model functions.
5	Model / DB	Model functions execute parameterised SQL queries against MySQL via the pool.
6	Response	JSON response is returned to the frontend and the UI is updated.

Authentication Flow

1. User submits email + password on `/login`.
2. Server fetches user by email; `bcrypt.compare()` checks password hash.
3. On success, `jwt.sign()` creates a 7-day token containing **id**, **email**, and **subscription_type**.
4. Token is returned to the frontend and stored in **localStorage**.
5. `AuthContext` reads the token on app load, validates expiry with `jwtDecode`, and sets the global user state.
6. All subsequent API requests attach the token via Axios interceptor.

7. On 401 responses, the Axios response interceptor clears localStorage and redirects to **/login**.

3. Backend

Project Structure

`server.js` — Entry point — Express app, middleware, route mounting, error handlers.

`config/db.js` — MySQL connection pool creation and debug connection test.

`routes/` — One router file per feature (`authRoutes`, `jobRoutes`, etc.).

`controllers/` — Business logic per feature — one function per endpoint.

`models/` — Direct SQL queries — each file maps to a DB table.

`middleware/authMiddleware.js` — JWT verification — attaches `req.user`.

`middleware/subscriptionMiddleware.js` — Reads subscription tier — attaches `req.subscription_type`.

`.env` — Environment variables (DB creds, JWT secret, Razorpay keys, PORT).

Server & Configuration

The server is a standard Express 5 app (ES Modules) with:

- `cors()` — allows cross-origin requests from the React dev server.
- `express.json()` — parses JSON request bodies.
- **Health check** GET / — returns a plaintext confirmation string.
- **404 handler** — catches all unmatched routes.
- **Global error handler** — 4-arg middleware logs and returns 500.

Database Models & Schema

users

Column	Type	Notes
<code>id</code>	<code>INT AUTO_INCREMENT PRIMARY KEY</code>	Primary key
<code>name</code>	<code>VARCHAR</code>	Display name
<code>email</code>	<code>VARCHAR UNIQUE</code>	Login identifier
<code>password</code>	<code>VARCHAR</code>	bcrypt hash (cost 10)
<code>subscription_type</code>	<code>ENUM('free', 'premium')</code>	Default: 'free'; set to 'premium' on payment

jobs

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key
user_id	INT FK → users.id	Owner of the job entry
company	VARCHAR	Company name
role	VARCHAR	Job title
work_type	VARCHAR	remote / onsite / hybrid
salary_range	VARCHAR	e.g. '10–15 LPA'
status	ENUM	saved / applied / interviewing / offer / rejected
date_applied	DATE	Application date
location	VARCHAR	City or 'Remote'
job_posting_url	VARCHAR	Link to job listing
notes	TEXT	Private notes
interview_questions	TEXT	Interview Q&A (also synced to community)
experience	TEXT	Interview experience narrative
share_in_community	TINYINT(1)	Flag: 1 = share post in community feed
created_at	TIMESTAMP	Auto-set on insert

community_posts

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key
user_id	INT FK → users.id	Author
job_id	INT FK → jobs.id	Source job entry (for sync/delete)
company	VARCHAR	Copied from job
role	VARCHAR	Copied from job
question	TEXT	Interview questions
experience	TEXT	Interview experience
created_at	TIMESTAMP	Auto-set on insert

resumes

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key
user_id	INT UNIQUE FK	One resume per user (upsert on conflict)

Column	Type	Notes
full_name	VARCHAR	
email	VARCHAR	
phone	VARCHAR	
linkedin	VARCHAR	Profile URL
skills	TEXT	Comma-separated or paragraph
education	JSON	Array of {degree, institution, year}
projects	JSON	Array of {name, description, link}
experience	JSON	Array of {company, role, duration, description}

payments

Column	Type	Notes
id	INT AUTO_INCREMENT PK	Primary key
user_id	INT FK → users.id	Paying user
razorpay_order_id	VARCHAR	Order ID from Razorpay API
razorpay_payment_id	VARCHAR	Payment ID (filled after verification)
amount	INT	Amount in paise (50000 = ₹500)
status	ENUM('pending' , 'paid')	Default 'pending'

Middleware

authMiddleware.js

Extracts the Bearer token from the Authorization header, verifies it with `jwt.verify(token, JWT_SECRET)`, and attaches the decoded payload to `req.user`. Returns 401 if the token is missing, expired, or tampered.

subscriptionMiddleware.js

A lighter middleware used only on the community route. It reads `req.user.subscription_type` (if a user is attached) and sets `req.subscription_type` to either 'premium' or 'free'. This allows the controller to decide the post limit without requiring authentication.

API Endpoints

Auth — /api/auth

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user. Validates fields, checks dupli
POST	/api/auth/login	None	Login. Compares bcrypt hash, returns JWT + us

Jobs — /api/jobs

Method	Endpoint	Auth	Description
GET	/api/jobs	JWT	Fetch all jobs for authenticated user, ordered by
POST	/api/jobs	JWT	Create job entry. If share_in_community=true, al
PUT	/api/jobs/:id	JWT	Update job. Syncs community_post: updates if e
DELETE	/api/jobs/:id	JWT	Delete job + its community_post (cascade clean)

Community — /api/community

Method	Endpoint	Auth	Description
GET	/api/community	Optional	Returns posts ordered by date DESC. Free user

Resume — /api/resume

Method	Endpoint	Auth	Description
GET	/api/resume	JWT	Fetch the authenticated user's saved resume row
POST	/api/resume	JWT	Upsert resume — INSERT ... ON DUPLICATE K

Payments — /api/payments

Method	Endpoint	Auth	Description
POST	/api/payments/create-order	JWT	Creates a Razorpay order for ₹500 (50000 paise)
POST	/api/payments/verify	JWT	Verifies HMAC-SHA256 signature. On success:

4. Frontend

Project Structure

`src/main.jsx` — React entry point — mounts App, injects global CSS.

`src/App.jsx` — BrowserRouter + AuthProvider wrapper. Defines all routes.

`src/services/api.js` — Axios instance with base URL + request/response interceptors.

`src/context/AuthContext.jsx` — Global auth state: user, login(), logout(), loading.

`src/components/Navbar.jsx` — Top navigation bar — adaptive for auth state and subscription tier.

`src/components/ProtectedRoute.jsx` — HOC that redirects unauthenticated users to /login.

`src/components/ResumePreview.jsx` — Read-only resume rendering component used for PDF export.

`src/pages/` — One file per page/route — 11 pages total.

`src/index.css` — Global Tailwind base + custom card/button utilities.

`src/resume.css` — Resume-specific print/layout styles for html2pdf export.

Routing & Page Map

Route	Component	Protected	Description
/	Home	No	Landing page with hero section and CTA.
/login	Login	No	Email + password form. Calls POST /api/auth/login, stores token.
/register	Register	No	Name + email + password. Calls POST /api/auth/register.
/dashboard	Dashboard	Yes	Stats grid (total, applied, interviewing, offers, rejected) + recent jobs list.
/jobs	Jobs	Yes	Full job list with edit/delete actions.
/add-job	AddJob	Yes	Form to create a new job entry with all fields.
/edit-job/:id	EditJob	Yes	Pre-filled edit form. Fetches job by ID on mount.
/community	Community	No	Interview Q&A feed. Free: 5 posts. Premium: unlimited.
/resume	Resume	Yes	Resume builder form + live preview + PDF download.
/upgrade	Upgrade	Yes	Razorpay checkout trigger for ₹500 upgrade.
/profile	Profile	Yes	Shows user name, email, and subscription badge.

Auth Context & State Management

The **AuthContext** wraps the entire application. It exposes **user**, **login()**, **logout()**, and **loading** to all components via `useContext(AuthContext)`.

- On mount, reads the JWT from `localStorage` and uses **jwtDecode** to check expiry.
- If expired: calls `logout()` which clears `localStorage` and redirects to `/login`.
- If valid: restores the user object from `localStorage` ('user' key).
- The **loading** flag prevents rendering `ProtectedRoute` before the check completes.
- **login(data)** stores token + user object and updates the state.
- **logout()** clears `localStorage`, nulls user state, and navigates to `/login`.

API Service Layer — `src/services/api.js`

A single Axios instance is created with `baseUrl: 'http://localhost:5000/api'`. Two interceptors are registered:

- **Request interceptor:** reads the JWT from `localStorage` and injects `Authorization: Bearer <token>` on every outgoing request.
- **Response interceptor:** catches 401 responses, clears `localStorage`, and redirects to `/login` — handling token expiry transparently.

Pages — Detail

Dashboard

- Fetches `GET /api/jobs` on mount.
- Derives 5 stats (total, applied, interviewing, offer, rejected) via `Array.filter`.
- Renders metric cards with color-coded counts.
- Shows the 5 most recent jobs with status badges.

Jobs

- Full list of user jobs. Each card shows company, role, status badge.
- Edit button navigates to `/edit-job/:id`.
- Delete button calls `DELETE /api/jobs/:id` and removes from local state.

AddJob / EditJob

- Large form with fields: `company`, `role`, `work_type`, `salary_range`, `status`, `date_applied`, `location`, `job_posting_url`, `notes`, `interview_questions`, `experience`, `share_in_community`.
- `EditJob` pre-fetches the job by ID and populates the form.
- `share_in_community` checkbox determines community post sync.

Community

- Public page — no auth required.

- Calls GET `/api/community`. Renders cards with company, role, question, experience.
- Unauthenticated or free users see a banner prompting upgrade for more posts.

Resume

- Split layout: form on left, live preview on right.
- Dynamic list sections for education, projects, experience (add/remove items).
- Save: POST `/api/resume`. Load: GET `/api/resume` on mount.
- Download: `html2pdf` captures the `#resume-preview` element and exports as A4 PDF.

Upgrade

- Calls POST `/api/payments/create-order` to get a Razorpay order.
- Opens Razorpay checkout modal with the order details.
- On payment success handler: calls POST `/api/payments/verify`.
- On verification: redirects to `/community`.

Components

Navbar — Renders logo + nav links + user section. Shows PRO/FREE badge on avatar. Hides Upgrade link for premium users. Collapses to login/register links when logged out.

ProtectedRoute — Wraps children in an auth check. If `loading=true`, renders nothing (prevents flicker). If no user, redirects to `/login` via React Router Navigate.

ResumePreview — Styled read-only resume layout rendering form state. Used both as a live preview and as the PDF export target (`#resume-preview`).

5. Subscription & Payments

Free vs Premium

Feature	Free	Premium (₹500)
Job tracking	Unlimited	Unlimited
Resume builder	Yes	Yes
Community posts visible	5 most recent	All posts
Dashboard	Full	Full
Profile badge	FREE	PRO

Razorpay Integration Flow

[Frontend] User clicks 'Upgrade ₹500' on the Upgrade page.

[Frontend → Backend] POST /api/payments/create-order (auth required).

[Backend] razorpay.orders.create({amount: 50000, currency: 'INR'}) is called. The order ID and amount are stored in the payments table with status='pending'.

[Backend → Frontend] Returns the Razorpay order object.

[Frontend] Opens Razorpay checkout modal with key, amount, and order_id. User completes payment.

[Razorpay → Frontend] On success, the handler receives razorpay_order_id, razorpay_payment_id, razorpay_signature.

[Frontend → Backend] POST /api/payments/verify with the three Razorpay fields.

[Backend] Recreates the expected HMAC-SHA256 signature: HMAC(order_id|payment_id, RAZORPAY_SECRET). Compares with received signature.

[Backend] On match: marks payment as 'paid', updates user.subscription_type to 'premium'.

[Frontend] Alerts '₹ Upgraded!' and redirects to /community.

6. Environment Variables

Backend — .env

Variable	Example	Purpose
PORT	5000	Express server port
DB_HOST	localhost	MySQL host
DB_USER	root	MySQL username
DB_PASSWORD	password	MySQL password
DB_NAME	jobify	MySQL database name
JWT_SECRET	supersecretkey	Secret for signing/verifying JWTs
RAZORPAY_KEY	rzp_test_xxx	Razorpay API key ID
RAZORPAY_SECRET	xxx	Razorpay API key secret (used for HMAC)

Frontend — .env

Variable	Example	Purpose
VITE_RAZORPAY_KEY	rzp_test_xxx	Razorpay key ID exposed to client for checkout modal

7. Setup & Running Locally

Prerequisites

- Node.js v18+
- MySQL 8+ running locally
- Razorpay test account (for payment features)

Backend Setup

1. **Extract** — Unzip backend.zip into a folder (e.g., jobify-backend/).
2. **Install** — Run: `npm install`
3. **Configure** — Copy `.env.example` to `.env` and fill in all variables (DB creds, JWT secret, Razorpay keys, `PORT=5000`).
4. **Database** — Create a MySQL database named `jobify`. Run the schema SQL to create tables: `users`, `jobs`, `community_posts`, `resumes`, `payments`.
5. **Start** — Development: `npm run dev` | Production: `npm start`
6. **Verify** — Visit `http://localhost:5000` — should return the health check message.

Frontend Setup

1. **Extract** — Unzip frontend.zip into a folder (e.g., jobify-frontend/).
2. **Install** — Run: `npm install`
3. **Configure** — Create `.env` in the root and set `VITE_RAZORPAY_KEY` to your Razorpay test key ID.
4. **API URL** — In `src/services/api.js`, ensure `baseUrl` matches your backend (default: `http://localhost:5000/api`).
5. **Start** — Run: `npm run dev` — Vite will serve on `http://localhost:5173`
6. **Razorpay script** — The Upgrade page uses `window.Razorpay` — ensure the Razorpay checkout.js CDN script is loaded in `index.html`.

Common Gotchas

- CORS errors: Make sure `cors()` is applied before routes in `server.js`.
- JWT mismatch: `JWT_SECRET` must be identical in `.env` and never change after users are created.

- Razorpay signature failure: RAZORPAY_SECRET (backend) and VITE_RAZORPAY_KEY (frontend) must use the same key pair.
- MySQL pool errors: Ensure DB_NAME database exists before starting the server.
- html2pdf blank export: Styles in resume.css must not use Tailwind's JIT — use standard CSS for print compatibility.
- Token not attached: Ensure localStorage key is exactly 'token' — the Axios interceptor reads that specific key.